

Project Quantum (Inventory Management System) - Technical Specification Document

This specification provides the comprehensive architecture, database schemas, containerization configuration, and business logic required to develop **Project Quantum** from scratch.

1. Project Concept & Requirements

Project Quantum is a secure, role-based enterprise inventory management system designed to track product quantities, audit stock modifications, manage supplier purchase orders, and trigger automated alerts when stock levels drop.

- **Core Requirements:**
- **Role-Based Access Control (RBAC):** Distinct permissions for roles (e.g., *Admin* has full CRUD, *Manager* can edit stock, *Staff* can only view stock and log movements).
- **Audit Trail:** Every change in stock quantity must log the operator, direction of change (**in, out, adjustment**), and description.
- **Threshold Alerting:** Whenever stock falls below a product's defined **min_threshold**, the system must dispatch asynchronous notification emails/Slack Webhooks to management.
- **Dockerization:** Deployable via Docker Compose with isolated web servers, databases, and background queue workers.

2. Tech Stack & Docker Setup

- **Framework:** PHP Laravel 10+ (using Eloquent ORM, Blade template engine, and Queue runner).
- **Database:** MySQL 8.
- **Cache & Queue Driver:** Redis.
- **Frontend:** Bootstrap 5 + Vanilla JavaScript.
- **Infrastructure:** Docker & Docker Compose.

Docker Environment Configuration (**docker-compose.yml**)

```
version: '3.8'

services:
```

```
app:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: quantum_app
  restart: unless-stopped
  working_dir: /var/www
  volumes:
    - ./var/www
  depends_on:
    - db
    - redis
  networks:
    - quantum-network

webserver:
  image: nginx:alpine
  container_name: quantum_webserver
  restart: unless-stopped
  ports:
    - "8080:80"
  volumes:
    - ./var/www
    - ./nginx.conf:/etc/nginx/conf.d/default.conf
  depends_on:
    - app
  networks:
    - quantum-network

db:
  image: mysql:8.0
  container_name: quantum_db
  restart: unless-stopped
  ports:
    - "3306:3306"
  environment:
    MYSQL_DATABASE: quantum_inventory
    MYSQL_ROOT_PASSWORD: secret_root_pass
    MYSQL_PASSWORD: secret_user_pass
    MYSQL_USER: quantum_user
  volumes:
    - dbdata:/var/lib/mysql
  networks:
    - quantum-network

redis:
  image: redis:alpine
  container_name: quantum_redis
  networks:
    - quantum-network

queue-worker:
  build:
    context: .
    dockerfile: Dockerfile
```

```
    container_name: quantum_queue_worker
    restart: unless-stopped
    command: php artisan queue:work --verbose --tries=3
    volumes:
      - ../var/www
    depends_on:
      - redis
    networks:
      - quantum-network

networks:
  quantum-network:
    driver: bridge

volumes:
  dbdata:
    driver: local
```

3. Database Schema Design (MySQL Migrations)

3.1 Users & Role-Based Auth Tables

```
// database/migrations/2026_01_01_000001_create_roles_table.php
Schema::create('roles', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique(); // e.g., admin, manager, staff
    $table->string('description')->nullable();
    $table->timestamps();
});

// database/migrations/2026_01_01_000002_create_role_user_table.php
Schema::create('role_user', function (Blueprint $table) {
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->foreignId('role_id')->constrained()->onDelete('cascade');
    $table->primary(['user_id', 'role_id']);
});
```

3.2 Inventory & Product Tables

```
// database/migrations/2026_01_01_000003_create_categories_table.php
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->text('description')->nullable();
```

```
        $table->timestamps();
    });

// database/migrations/2026_01_01_000004_create_products_table.php
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->foreignId('category_id')->constrained()->onDelete('restrict');
    $table->string('sku')->unique();
    $table->string('name');
    $table->text('description')->nullable();
    $table->decimal('price', 10, 2);
    $table->integer('quantity')->default(0);
    $table->integer('min_threshold')->default(10); // Trigger limit
    $table->timestamps();
});
```

3.3 Supplier & Audit Trail Log Tables

```
// database/migrations/2026_01_01_000005_create_suppliers_table.php
Schema::create('suppliers', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('phone');
    $table->text('address');
    $table->timestamps();
});

// database/migrations/2026_01_01_000006_create_stock_logs_table.php
Schema::create('stock_logs', function (Blueprint $table) {
    $table->id();
    $table->foreignId('product_id')->constrained()->onDelete('cascade');
    $table->foreignId('user_id')->constrained(); // Operator
    $table->integer('quantity_changed'); // e.g., +20 or -5
    $table->enum('type', ['in', 'out', 'adjustment']);
    $table->string('reason'); // e.g., "Received purchase order #12"
    $table->timestamp('created_at')->useCurrent();
});
```

4. Role-Based Access Control (RBAC) Logic

4.1 Custom Middleware for Roles

```
// app/Http/Middleware/CheckRole.php
```

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class CheckRole
{
    public function handle(Request $request, Closure $next, ...$roles)
    {
        if (!$request->user() || !$request->user()->hasAnyRole($roles)) {
            abort(403, 'Unauthorized access.');
```

4.2 Helper Method in User model

```
// app/Models/User.php
public function roles() {
    return $this->belongsToMany(Role::class);
}

public function hasAnyRole(array $roles) {
    return $this->roles()->whereIn('name', $roles)->exists();
}
```

5. Stock Threshold & Alert Engine (Events/Queues)

When stock level adjusts, we trigger audits and queue a task if stock drops below threshold.

5.1 Triggering Event inside Controller/Service

```
namespace App\Services;

use App\Models\Product;
use App\Models\StockLog;
use App\Events\StockLevelChanged;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Auth;

class StockService
{
    public function adjustStock(Product $product, int $quantityChange, string $type, string $reason)
```

```

    {
        DB::transaction(function () use ($product, $quantityChange, $type, $reason) {
            $product->quantity += $quantityChange;
            $product->save();

            StockLog::create([
                'product_id' => $product->id,
                'user_id' => Auth::id(),
                'quantity_changed' => $quantityChange,
                'type' => $type,
                'reason' => $reason
            ]);

            // Dispatch Stock Check Event
            event(new StockLevelChanged($product));
        });
    }
}

```

5.2 StockLevelChanged Event

```

// app/Events/StockLevelChanged.php
namespace App\Events;

use App\Models\Product;
use Illuminate\Foundation\Events\Dispatchable;
use Illuminate\Queue\SerializesModels;

class StockLevelChanged
{
    use Dispatchable, SerializesModels;

    public $product;

    public function __construct(Product $product)
    {
        $this->product = $product;
    }
}

```

5.3 Event Listener checks Threshold and dispatches Queue Job

```

// app/Listeners/CheckStockThreshold.php
namespace App\Listeners;

use App\Events\StockLevelChanged;
use App\Jobs\SendLowStockAlertEmail;

class CheckStockThreshold
{

```

```
    public function handle(StockLevelChanged $event)
    {
        $product = $event->product;
        if ($product->quantity <= $product->min_threshold) {
            // Queue the job to dispatch asynchronously
            SendLowStockAlertEmail::dispatch($product);
        }
    }
}
```

5.4 Low Stock Email Asynchronous Job

```
// app/Jobs/SendLowStockAlertEmail.php
namespace App\Jobs;

use App\Models\Product;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Mail;

class SendLowStockAlertEmail implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    protected $product;

    public function __construct(Product $product)
    {
        $this->product = $product;
    }

    public function handle()
    {
        // Execute mail dispatch or Slack webhook trigger
        Mail::to('manager@example.com')->send(new \App\Mail\LowStockAlertMail($this->product));
    }
}
```

6. Frontend Layout & Blades

1. **Dashboard Overview:** Summary counters for **Total SKU**, **Low Stock Items**, **Pending Purchase Orders**, and **Activity log list** in a responsive Bootstrap table.

2. **Product Directory:** Data grids with conditional styling: products under threshold display with a soft red badge (`class="badge bg-danger"`).
 3. **Adjustment Modals:** Operates direct adjustments with secure input logging (plus/minus changes).
-

7. Step-by-Step Development Phases

Phase 1: Docker Scaffold & Project Setup

1. Compose Nginx, PHP, MySQL, and Redis services via Docker.
2. Verify connections and build project via Composer.
3. Configure `.env` variables matching the Docker services structure.

Phase 2: Migrations & Authentication

1. Generate MySQL tables structure with appropriate keys and indexes.
2. Build default Roles (Admin, Manager, Staff) via seeders.
3. Configure built-in Laravel session authentication.

Phase 3: Stock Movement & Logging Logic

1. Implement the Product inventory CRUD.
2. Establish `StockService` to handle adjustments inside DB Transactions and store logs.
3. Restrict controller actions using standard middleware rules (`CheckRole`).

Phase 4: Async Alert Engine

1. Configure Laravel queue connection to run on Redis.
2. Code the events, queue-workers, and SMTP mail configuration templates.
3. Launch queue worker container and test delivery of background mails.

Phase 5: Verification & End-to-End Tests

1. Authenticate as a Staff user. Adjust stock of "Item X" (stock is 15, threshold is 10) down by 6.
2. Verify DB: Stock level is 9. A record in `stock_logs` exists with correct timestamps and author.

3. Verify Queue logs: Redis queue received job, **quantum_queue_worker** executed, and mock SMTP server catches the low stock warning mail.